

Evaluation of Terraform Security Scanners and Policy-as-Code Gates for AWS IaC Misconfiguration Detection

Vishvaksen Machana
25173421
MSc in Cloud Computing

12th June 2026

1 Research Problem Background

To gain insight into how much developers actually practice security, citetverdet2025 conduct a study of 812 open-source Terraform projects on AWS, Azure and Google Cloud by running the static analyzers Checkov and tfsec to check for security in the infrastructure, finding that there are inconsistent practices across the projects, with encryption at rest being the least practiced category. Their analysis is based solely on the scanners—they are not used to measure the accuracy with which those scanners (or gates in a policy-as-code pipeline) detect misconfigurations against a set of ground truth; nor is it used to establish whether enforcing those scanners (or gates) in a pipeline measurably reduces risk.

With that said, no one tool does it all: Checkov and KICS find that they miss context-specific misconfigurations in real charts, and Toprani and Madisetti (2025) determine that static infrastructure-as-code scanners fail to detect context-specific misconfigurations in cloud templates. What is not measured and is discussed here, is the accuracy of Terraform scanners and policy-as-code gates in detecting AWS misconfigurations with labels and what risk reduction they provide at different stages of the pipeline.

2 Research Question

To what extent do Terraform security scanners and policy-as-code gates accurately detect labelled AWS infrastructure-as-code misconfigurations and reduce deployment risk in a CI pipeline?

3 Justification

One of the most common ways to compromise the cloud is by misconfiguration; teams are adding scanners and policy gates onto their pipelines in ever increasing numbers, but they are doing so with little evidence of their effectiveness, in terms of the amount detected and the amount of risk eliminated. This research extends the adoption results of Verdet et al. (2025) to a controlled detection benchmark on AWS. The secure and insecure version of a common AWS resource are generated for each category of misconfiguration (e.g., using AWS S3, misconfiguration is either S3Resource or S3AccessPolicy) and each are stored with the misconfiguration type and the expected outcome (e.g., S3Resource and S3AccessPolicy). The identical cases are analyzed by Checkov, tfsec, and custom Open Policy Agent and Rego rules in a continuous integration workflow, and the results of the three enforcement stages – manual review, static scanning and an enforced policy gate – are contrasted. Measured outcomes include detection precision, recall and F1, false positive and false negative, severity coverage, scan time and remediation effort in changed lines, which in turn are used to calculate the risk reduction of each pipeline stage, defined as the percentage drop in high- and medium-severity misconfigurations that have not been detected after each stage. Each snippet is synthetic, so only safe resources are ever used in the researcher’s account, and no production or personal data is being used, so there is minimal ethical risk and it’s within the provider’s free-tier allowance.

4 Specific Items to Be Addressed

1. *Labelled dataset*: examine the common AWS misconfiguration categories (public storage, over-permissive access, unencrypted data, weak logging) and build secure and insecure Terraform modules labelled by category and expected outcome.
2. *Tool comparison*: study Checkov, tfsec and custom Open Policy Agent and Rego policies against the same labelled cases.
3. *Detection accuracy*: measure precision, recall, F1, false positives, false negatives and severity coverage for each tool.
4. *Pipeline impact*: compare manual review, static scanning and an enforced continuous-integration policy gate for detection and risk reduction.
5. *Cost*: measure scan time and remediation effort so detection effectiveness is weighed against pipeline overhead.
6. *Guidance*: analyse which checks deliver measurable risk reduction and derive a minimum policy set for small AWS teams and student labs.

Bibliography

- Minna, F., Massacci, F. and Tuma, K. (2025) ‘Analyzing and mitigating (with LLMs) the security misconfigurations of Helm charts from Artifact Hub’, *Empirical Software Engineering*, 30(5), 132. doi: 10.1007/s10664-025-10688-0.
- Toprani, D. and Madiseti, V.K. (2025) ‘LLM agentic workflow for automated vulnerability detection and remediation in infrastructure-as-code’, *IEEE Access*, 13, 69175-69181. doi: 10.1109/ACCESS.2025.3560911.
- Verdet, A., Hamdaqa, M., Da Silva, L. and Khomh, F. (2025) ‘Assessing the adoption of security policies by developers in Terraform across different cloud providers’, *Empirical Software Engineering*, 30(3), 74. doi: 10.1007/s10664-024-10610-0.